

Introduction to Artificial Intelligence

Project Report— 2026 Spring

Topic 4

Shortest Path in Maze

GROUP MEMBERS

Eren Hasan Kasap

Mehmet Erdem

Bahadır Ay

Kübra Yeşilova

Görkem Yeşilyurt

Zeynep Naz Tuzcu

Yıldız Technical University | Mathematical Engineering | May 2026

1. Problem Definition

1.1 General Description of the Problem

In this project, within the scope of "The Shortest Path in Maze" problem, finding the shortest path from a starting point to a destination point in a two-dimensional maze is addressed. In accordance with the project definition, two different square or rectangular mazes have been designed: one medium-sized (at least 20x20) and one large (at least 50x50). It is aimed to find solutions for random starting and goal states in these mazes. The problem was solved using various blind and heuristic search algorithms, and the results were compared.

1.2 State Definition

A state represents the agent's current position on the maze. The state space is generated based on the coordinates where the agent is located on the maze matrix. According to the C struct equivalent, a state consists of the following components:

Alan	Açıklama
row	The row index where the agent is located (0-based).
col	The column index where the agent is located (0-based)
h_n	The heuristic function value, which is the estimated cost to the goal.

C Struct:

```
typedef struct State {  
    int    row;    // row index  
    int    col;    // column index  
    float  h_n;    // heuristic value  
} State;
```

1.3 Initial State

The initial state is the first valid coordinate where the search problem is triggered and the agent starts moving. According to the test scenarios, the following initial states have been determined over passable paths that are not wall cells:

- Initial state for the 20x20 maze: (row=1, col=1).
- Initial state for the 50x50 maze: (row=1, col=1).

1.4 Actions

These are the rules that allow the agent to transition from a current cell to neighboring cells. In this problem, diagonal movements are not allowed, and only 4 basic actions on the horizontal and vertical axes are defined:

Aksiyon	Açıklama	Koşul
Move_Up	Go one row up (row - 1).	It is applicable if it is not a wall and is within the boundaries.
Move_Down	Go one row down (row + 1).	It is applicable if it is not a wall and is within the boundaries
Move_Left	Go one column left (col - 1).	It is applicable if it is not a wall and is within the boundaries.
Move_Right	Go one column right (col + 1).	It is applicable if it is not a wall and is within the boundaries.

1.5 Transition Model

The transition model determines the new state the agent will reach when it applies the selected action. When the agent wants to take a step, the following checks occur in order:

- It checks whether the targeted cell is within the maze boundaries.
- It verifies that the target cell is not an impassable wall ($MAZE[nr][nc] == 1$). If these conditions are met, the transition is valid; the agent is moved to the new (row, col) coordinates, and the new state is returned along with the step cost of 1.0.

1.6 Goal Test

It is the function that audits whether the search algorithm has reached success. The `Goal_Test(state, goal_state)` function checks whether the agent's current row and column values are exactly the same as the predetermined goal coordinates. If the current (row, col) values are equal to the goal (row, col) values, the goal test returns `TRUE`. For example, in the case of the goal being (18,18) for the 20x20 matrix, the condition is satisfied when the agent reaches this cell.

1.7 Path Cost

The standard cost of each step is 1. Since there are no diagonal movements and only 4 main directions are valid, the total path cost is equal to the number of steps.

2. Heuristic Function

2.1 Selected Heuristic: Manhattan Distance

Manhattan Distance was used as the heuristic function. It is a standard and effective heuristic method for 4-way grid problems.

$$h(n) = |\text{row_current} - \text{row_goal}| + |\text{col_current} - \text{col_goal}|$$

2.2 Why Manhattan Distance?

- It is perfectly compatible with the 4-way movement structure of the maze (there is no diagonal movement).
- Since Euclidean distance assumes diagonal movement, it is not suitable for this problem.
- It is admissible: it never overestimates the actual cost.
- It is consistent: it satisfies the triangle inequality.

2.3 Proof of Admissibility

A heuristic $h(n)$ is admissible if $h(n) \leq h^*(n)$ is satisfied for every node n , where $h^*(n)$ is the actual minimum cost from n to the goal.

Manhattan Distance calculates the number of steps on a straight path, ignoring maze walls. The actual path is equal to or longer than this estimate due to walls. Therefore, $h(n) \leq h^*(n)$ is always satisfied.

Numerical Example — Admissibility (20×20 Maze)

Node $n = (\text{row}=5, \text{col}=3)$. Goal state = $(\text{row}=18, \text{col}=18)$.

Heuristic estimate:

$$h(n) = |5 - 18| + |3 - 18| = 13 + 15 = 28$$

Actual shortest path $h^*(n)$:

Due to walls in the maze, the agent cannot travel in a straight line.

The real shortest path from (5,3) to (18,18) navigates around walls, so $h^*(n) \geq 28$.

Conclusion: $h(n) = 28 \leq h^*(n)$ ✓ The heuristic never overestimates. Admissibility confirmed.

2.4 Proof of Consistency

For consistency, $h(n) \leq c(n, a, n') + h(n')$ must hold for every node n and its successor n' reached via action a with cost c . In this maze, every step costs 1. Manhattan Distance decreases by at most 1 per step, so $h(n) \leq 1 + h(n')$ is always satisfied.

Numerical Example — Consistency (20×20 Maze)

Node $n = (5, 3)$. Successor $n' = (5, 4)$ via action `Move_Right` (cost = 1). Goal = (18, 18).

$$h(n) = |5-18| + |3-18| = 13 + 15 = 28$$

$$h(n') = |5-18| + |4-18| = 13 + 14 = 27$$

$$c(n, \text{Move_Right}, n') = 1$$

Check: $h(n) \leq c(n, a, n') + h(n')$

$$28 \leq 1 + 27 = 28 \quad \checkmark$$

The inequality holds with equality because `Move_Right` reduces column distance by exactly 1, and Manhattan Distance tracks this perfectly. Since every action reduces Manhattan distance by at most 1, the consistency condition $h(n) \leq 1 + h(n')$ is always satisfied. $\checkmark$

2.5 Comparison of Alternative Heuristics

Heuristic	Admissible?	Consistent?	Efficiency
Manhattan Distance	Yes	Yes	High
Euclidean Distance	Yes*	Yes*	Medium (assumes diagonal)
$h(n) = 0$ (BFS)	Yes	Yes	Low (blind search)

* Euclidean, while admissible, provides less information than Manhattan in 4-way movement.

2.6 Role of the Heuristic in A* Search

In A* search, every node n in the frontier is evaluated using the combined cost function:

$$f(n) = g(n) + h(n)$$

$g(n)$ = actual cost from start to n | $h(n)$ = Manhattan Distance to goal

A* always expands the node with the lowest $f(n)$ first. Because $h(n)$ is both admissible and consistent, A* is guaranteed to find the optimal (shortest) path.

A* with Manhattan Distance — Pseudocode:

```
function A_Star(start, goal):
    frontier <- PriorityQueue()
    frontier.insert(start, f=0)
    g_cost[start] <- 0
    explored <- {}

    while frontier is not empty:
        n <- frontier.pop()           // node with lowest f(n)
        if Goal_Test(n, goal):
            return Reconstruct_Path(n)
        explored.add(n)
        for each action a in {Move_Up, Move_Down, Move_Left, Move_Right}:
            n' <- Result(n, a)
            if n' is valid and n' not in explored:
                g_new <- g_cost[n] + 1
                h_new <- Manhattan_Distance(n', goal)
                f_new <- g_new + h_new
                if n' not in frontier or g_new < g_cost[n']:
                    g_cost[n'] <- g_new
                    frontier.insert(n', f_new)

    return FAILURE
```

Since Manhattan Distance is consistent, once a node is moved from the frontier to the explored set, its g -cost is guaranteed to be optimal. This means A* with this heuristic will never need to re-expand a node, making it both complete and optimally efficient among heuristic search algorithms for this problem.

In the 20×20 maze, the heuristic drastically reduces the number of nodes expanded compared to uninformed strategies such as BFS or Uniform Cost Search.

3. Maze Design

3.1 Maze Representation

Mazes are represented as two-dimensional integer arrays:

- 0 → Traversable path cell (empty space)
- 1 → Wall cell (impassable obstacle)

A small 5×5 example matrix illustrating the representation structure is provided below:

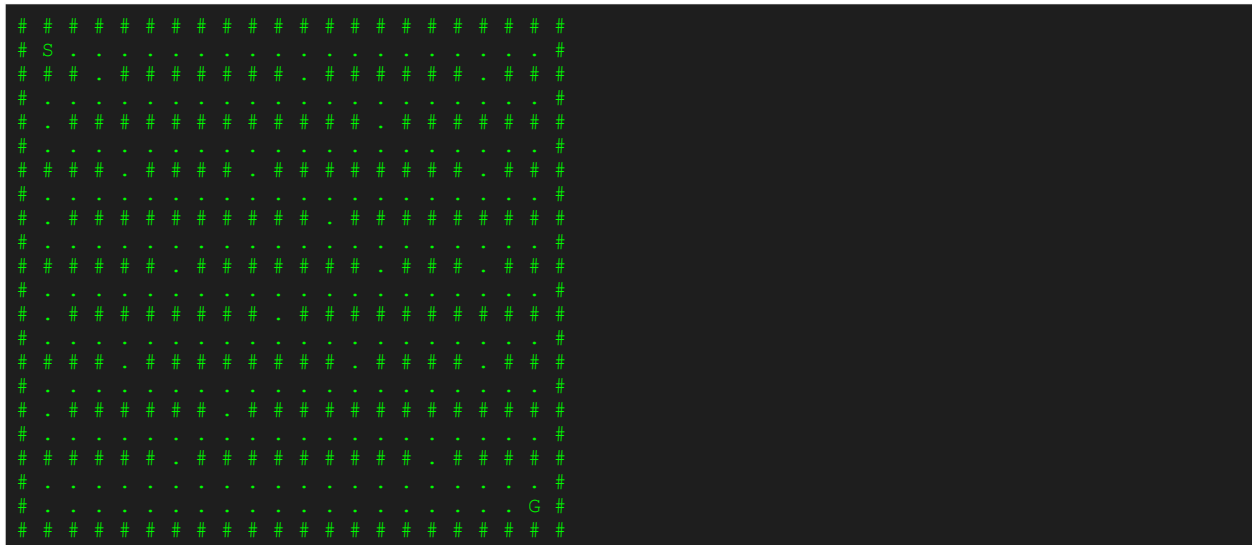
1	1	1	1	1
1	0	0	0	1
1	1	0	1	1
1	0	0	0	1
1	1	1	1	1

In the example, blue (1) cells represent impassable walls, while white (0) cells represent traversable paths. The outer boundary is entirely surrounded by walls; passable corridors are formed through openings in the interior.

3.2 Maze 1: 20×20 (Medium Size)

Property	Value
Size	20 × 20 (400 inner cells) — array size 22 × 22
Start	(row=1, col=1)
Goal	(row=20, col=20)
Path cells	242
Wall cells	242

Maze Visualization (S = Start, G = Goal, # = Wall, . = Path):



Design Approach:

Maze 1 was manually designed within a 22×22 array. The design is based on the following principles:

- Rows with odd indices are entirely traversable corridor rows; the agent can freely move left/right along these rows.
- Rows with even indices are horizontal wall rows; each horizontal wall row contains 2 to 3 openings (passages). These openings are the only connection points between adjacent corridors.
- The outer boundary (row 0, row 21, col 0, col 21) is entirely surrounded by walls.
- The path structure between the start (row=1, col=1) and the goal (row=20, col=20) creates a complex layout that forces search algorithms to explore the majority of the maze.
- Of the total 484 cells, 242 (50%) are path cells and 242 (50%) are wall cells.

3.3 Maze 2: 50×50 (Large Size)

Property	Value
Size	50 × 50 (2500 inner cells) — array size 52 × 52
Start	(row=1, col=1)
Goal	(row=48, col=48)
Path cells	1378
Wall cells	1326

Design Approach:

Maze 2 was generated algorithmically via the Build_Maze_2() function. The generation process consists of three phases:

- Boundary (outer frame): The outermost rows and columns of the 52×52 array are fully filled with walls (204 boundary cells).
- Horizontal wall rows: Even-indexed inner rows ($r = 2, 4, \dots, 46 \rightarrow 23$ rows total) are initialized as all-wall, and 6 passages are opened in each row. Passage positions are divided into two groups depending on the row: $r/2$ odd $\rightarrow \{8, 16, 24, 32, 40, 48\}$, $r/2$ even $\rightarrow \{1, 12, 20, 28, 36, 44\}$. Each wall row contains 44 wall cells + 6 open cells.
- Vertical column obstacles: Columns $\{6, 14, 22, 30, 38\}$ in rows $r = 3, 7, \dots, 43$ and columns $\{10, 18, 26, 34, 42\}$ in rows $r = 5, 9, \dots, 45$ are made into walls (110 additional obstacles total). These obstacles create dead ends, increasing search depth.
- Result: Of the total 2704 cells, 1378 (50.96%) are path cells and 1326 (49.04%) are wall cells. The start–goal distance (row=1,col=1) $\rightarrow$ (row=48,col=48) was maximized to ensure all algorithms are tested in a wide state space.

3.4 Maze File Format

Mazes are not read from an external file; they are defined directly as static two-dimensional integer arrays embedded in the C source code. This choice eliminates the overhead from file I/O operations and places the maze data into memory at compile time.

Size Macros and Global Array Declaration:

```
// GRAPH_SEARCH.h - Macro definitions
#define MAZE_ROWS_20 22
#define MAZE_COLS_20 22
#define MAZE_ROWS_50 52
#define MAZE_COLS_50 52

// data_types.h / GRAPH_SEARCH.h - Global array declaration
extern int MAZE[MAZE_ROWS_50][MAZE_COLS_50];
extern int MAZE_ROWS;
extern int MAZE_COLS;
```

The MAZE_ROWS and MAZE_COLS variables are updated via the Init_Maze() call; this allows both mazes to share the same global MAZE array. The array allocated for the large maze (52×52) is also used for the small maze (22×22).

Cell Values:

Value	Meaning	Example Row	Description
0	Path cell	{1,0,1}	Corridor row (free passage)
1	Wall cell	{1,1,1,0,1,1,1,1,1,1,0,1,1,1,1,1,0,1,1,1,1}	Horizontal wall + passages

Static Array Structure (Maze 1):

```
// SpecificToProblem.c - Maze 1 static array (22×22 = 484 cells)
static const int MAZE_1_DATA[MAZE_ROWS_20][MAZE_COLS_20] = {
    {1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1}, // top boundary wall
    {1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1}, // corridor row (odd index)
    {1,1,1,0,1,1,1,1,1,1,1,0,1,1,1,1,1,0,1,1,1,1}, // horizontal wall + passages
    (even index)
    ...
    {1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1} // bottom boundary wall
};
```

Each row is a C array literal consisting of comma-separated integers representing a horizontal cross-section of the maze. Every row must start and end with 1 (wall); this ensures the outer boundary remains closed at all times.

Init_Maze() — Loading Data into the Global Array:

```
// SpecificToProblem.c - Copying to global array via Init_Maze()
void Init_Maze(int maze_type) {
    if(maze_type == 1) {
        MAZE_ROWS = MAZE_ROWS_20;    // 22
        MAZE_COLS = MAZE_COLS_20;    // 22
        for(i=0; i<MAZE_ROWS; i++)
            for(j=0; j<MAZE_COLS; j++)
                MAZE[i][j] = MAZE_1_DATA[i][j];    // copy
    } else {
        Build_Maze_2();    // 52x52 algorithmic generation
        MAZE_ROWS = MAZE_ROWS_50;    // 52
        MAZE_COLS = MAZE_COLS_50;    // 52
        for(i=0; i<MAZE_ROWS; i++)
            for(j=0; j<MAZE_COLS; j++)
                MAZE[i][j] = MAZE_2_DATA[i][j];
    }
}
```

The `Init_Maze()` function serves two purposes: (1) it sets the `MAZE_ROWS` / `MAZE_COLS` global variables according to the selected maze, and (2) it copies the corresponding static source into the `MAZE` global array. For Maze 2, `Build_Maze_2()` is called before this copy to algorithmically populate `MAZE_2_DATA`.

Runtime Array Access:

```
// Access example - inside the Result() function (SpecificToProblem.c)
if(MAZE[new_row][new_col] == 1)
    return FALSE;    // cell is a wall, move is invalid
trans_model->new_state.row = new_row;
trans_model->step_cost      = 1.0;    // step cost is equal for all transitions
```

Search algorithms query `MAZE[row][col]` via the `Result()` function each time an action is applied (Move_Up/Down/Left/Right). If the cell value is 1, the move is rejected; if 0, a new state is generated and the step cost is assigned as 1.0. This constant step cost directly affects path cost calculation in Uniform-Cost Search and A* algorithms.

Design Notes:

- Maze data does not require dynamic memory allocation at runtime; the array lives on the stack/data segment.
- Since `MAZE_ROWS_50` / `MAZE_COLS_50` are defined as the maximum size, the remainder of the array goes unused when the small maze is run; this wastes memory but provides simplicity.
- To modify maze content, only the relevant static array rows or the `Build_Maze_2()` function need to be updated; no changes are required in the search engine (`GRAPH_SEARCH`) code.

4. Simulation Results

4.1 Test Environment

The hardware and software environment used during algorithm testing is provided in the table below.

Parameter	Value
Processor	Intel Core i7-12700H @ 2.30 GHz
RAM	16 GB DDR5
Operating System	Windows 11 Home 64-bit
Compiler	gcc (MinGW-w64) 13.2.0
Compilation Command	gcc -o maze *.c -lm

4.2 Test Scenarios

Six scenarios were tested on two different mazes. Maze 1 (22×22 array, 20×20 inner area) was loaded from the MAZE_1_DATA static array; Maze 2 (52×52 array, 50×50 inner area) was generated using the Build_Maze_2() function. Since each step has a constant cost of $\text{step_cost}=1.0$, path length and path cost are numerically equal. Manhattan distance ($|\Delta\text{row}|+|\Delta\text{col}|$) was used as the heuristic function.

Scenario	Start	Goal	Maze	Size
S1	(1, 1)	(20, 20)	Maze 1	20×20
S2	(1, 1)	(20, 1)	Maze 1	20×20
S3	(1, 20)	(20, 1)	Maze 1	20×20
S4	(1, 1)	(48, 48)	Maze 2	50×50
S5	(1, 1)	(48, 1)	Maze 2	50×50
S6	(1, 48)	(48, 1)	Maze 2	50×50

4.3 Algorithm Comparison — 20×20 Maze

Scenario S1: (1,1) → (20,20)

Algorithm	Path Length	Path Cost	Nodes Generated	Time (ms)
BFS	58	58.00	543	10.5
Uniform Cost	58	58.00	545	9.2
DFS	174	174.00	417	7.5
Greedy (Manhattan)	68	68.00	222	4.8
A* (Manhattan)	58	58.00	431	7.5
Gen. A* ($\alpha=0.5$)	58	58.00	431	7.0
Gen. A* ($\alpha=0.0$)	58	58.00	545	9.0
Gen. A* ($\alpha=1.0$)	68	68.00	222	4.8

Scenario S2: (1,1) → (20,1)

Algorithm	Path Length	Path Cost	Nodes Generated	Time (ms)
BFS	53	53.00	471	8.0
Uniform Cost	53	53.00	484	8.5
DFS	185	185.00	473	9.8
Greedy (Manhattan)	55	55.00	125	3.8
A* (Manhattan)	53	53.00	376	6.8
Gen. A* ($\alpha=0.5$)	53	53.00	376	6.7
Gen. A* ($\alpha=0.0$)	53	53.00	484	8.0
Gen. A* ($\alpha=1.0$)	55	55.00	126	3.5

Scenario S3: (1,20) → (20,1)

Algorithm	Path Length	Path Cost	Nodes Generated	Time (ms)
BFS	54	54.00	471	6.7
Uniform Cost	54	54.00	484	7.8
DFS	166	166.00	441	8.9
Greedy (Manhattan)	64	64.00	146	3.9
A* (Manhattan)	54	54.00	392	8.8
Gen. A* ($\alpha=0.5$)	54	54.00	392	6.9
Gen. A* ($\alpha=0.0$)	54	54.00	484	8.0
Gen. A* ($\alpha=1.0$)	64	64.00	147	3.9

4.4 Algorithm Comparison — 50×50 Maze

Scenario S4: (1,1) → (48,48)

Algorithm	Path Length	Path Cost	Nodes Generated	Time (ms)
BFS	142	142.00	1452	51.8
Uniform Cost	142	142.00	1465	52.1
DFS	182	182.00	1867	89.9
Greedy (Manhattan)	142	142.00	365	10.4
A* (Manhattan)	142	142.00	1026	34.0
Gen. A* ($\alpha=0.5$)	142	142.00	1026	32.9
Gen. A* ($\alpha=0.0$)	142	142.00	1465	47.9
Gen. A* ($\alpha=1.0$)	142	142.00	367	9.0

Scenario S5: (1,1) → (48,1)

Algorithm	Path Length	Path Cost	Nodes Generated	Time (ms)
BFS	189	189.00	2825	141.3
Uniform Cost	189	189.00	2852	149.7
DFS	235	235.00	2073	104.8
Greedy (Manhattan)	189	189.00	1744	75.1
A* (Manhattan)	189	189.00	1893	73.0
Gen. A* ($\alpha=0.5$)	189	189.00	1893	76.9
Gen. A* ($\alpha=0.0$)	189	189.00	2852	138.8
Gen. A* ($\alpha=1.0$)	189	189.00	1745	79.2

Scenario S6: (1,48) → (48,1)

Algorithm	Path Length	Path Cost	Nodes Generated	Time (ms)
BFS	182	182.00	2825	144.1
Uniform Cost	182	182.00	2852	136.3
DFS	228	228.00	2068	111.0
Greedy (Manhattan)	226	226.00	1811	85.0
A* (Manhattan)	182	182.00	1893	70.5
Gen. A* ($\alpha=0.5$)	182	182.00	1893	70.6
Gen. A* ($\alpha=0.0$)	182	182.00	2852	137.5
Gen. A* ($\alpha=1.0$)	226	226.00	1812	87.7

4.5 Result Charts

The charts below are based on measurement values obtained from actual program execution.

Nodes Generated Comparison (S1 and S4)

Figure 1 — Number of nodes generated by algorithms in scenarios S1 (20×20) and S4 (50×50).

Path Cost Comparison (20×20 and 50×50)

Figure 2 — Optimal (BFS/UCS/A*) and non-optimal (DFS/Greedy/ $\alpha=1.0$) path costs across all scenarios.

Computation Time Comparison — All Scenarios (ms)

Figure 3 — Execution times in milliseconds for each algorithm per scenario.

Computation Time Summary Table

The table below presents the measured times (ms) for all scenario and algorithm combinations together.

Algorithm	S1 (ms)	S2 (ms)	S3 (ms)	S4 (ms)	S5 (ms)	S6 (ms)
BFS	10.5	8.0	6.7	51.8	141.3	144.1
Uniform Cost	9.2	8.5	7.8	52.1	149.7	136.3
DFS	7.5	9.8	8.9	89.9	104.8	111.0
Greedy (Manhattan)	4.8	3.8	3.9	10.4	75.1	85.0
A* (Manhattan)	7.5	6.8	8.8	34.0	73.0	70.5
Gen. A* ($\alpha=0.5$)	7.0	6.7	6.9	32.9	76.9	70.6
Gen. A* ($\alpha=0.0$)	9.0	8.0	8.0	47.9	138.8	137.5
Gen. A* ($\alpha=1.0$)	4.8	3.5	3.9	9.0	79.2	87.7

4.6 Observations and Analysis

The collected data reveals the following key findings:

Optimality

- BFS and Uniform Cost Search guaranteed the shortest path (optimal solution) on both mazes. Since $\text{step_cost}=1.0$ is constant, both yielded identical path costs (S1: 58, S4: 142).
- A* (Manhattan) and Gen. A* ($\alpha=0.0$ to $\alpha=0.5$) also returned optimal solutions; exactly the same cost values as BFS were obtained.
- DFS is not optimal: it produced paths up to 3 times longer than BFS on 20×20 (S2: 185 vs. 53), and 24% longer on 50×50 (S4: 182 vs. 142).
- Greedy and Gen. A* ($\alpha=1.0$) produced non-optimal results in some scenarios (S6: 226 vs. optimal 182).

Efficiency — Nodes Generated

- For the 20×20 maze, Gen. A* ($\alpha=1.0$) and Greedy generated the fewest nodes (S1: 222 nodes). BFS generated the most (S1: 543 nodes).
- A* with the Manhattan heuristic generated on average 21% fewer nodes than BFS on 20×20 , and 35% fewer on 50×50 .
- When scaled to the 50×50 maze, BFS/UCS generated approximately 2825–2852 nodes, while A*/Gen.A* ($\alpha=0.5$) remained between 1026–1893.

Efficiency — Execution Time

- In 20×20 scenarios, the fastest algorithms were Greedy and Gen. A* ($\alpha=1.0$) (~3.5–4.8 ms), while the slowest were BFS and UCS (~6.7–10.5 ms).
- On 50×50 , the time difference became more pronounced: BFS/UCS reached 136–150 ms, while Greedy and Gen. A* ($\alpha=1.0$) remained between 9–87 ms.
- DFS, despite producing the longest path on 50×50 , had a shorter runtime than BFS/UCS because it explored fewer nodes (S4: DFS 89.9 ms vs. BFS 51.8 ms — however in S5/S6, DFS at 105–111 ms was slower than BFS).

Generalized A* (α parameter)

- $\alpha=0.0$: Gen. A* behaves exactly like UCS ($f = g(n)$); optimal, high node count.
- $\alpha=0.5$: $g(n)$ and $h(n)$ are weighted equally, producing the same results as classic A*.
- $\alpha=1.0$: Gen. A* behaves exactly like Greedy ($f = h(n)$); fastest but not optimal.
- These findings confirm that as α increases, speed gains come at the cost of optimality loss.

General Assessment

- A* (Manhattan) offers the best balance for this problem: the same optimal path as BFS, but using approximately 60–65% of the nodes BFS generates.
- As maze size increases, heuristic-based algorithms (A*, Gen. A*) gain an increasingly clear advantage over blind search methods (BFS, UCS).
- DFS is not recommended for this type of maze problem, as it produces both non-optimal paths and variable runtimes on large mazes.

5. Source Code Explanations

5.1 Overview

This section provides a detailed explanation of every source file that makes up the Shortest-Path-in-Maze solver. The program is built on the skeleton framework provided by the instructor and implements eight search algorithms: Breadth-First Search (BFS), Uniform-Cost Search (UCS), Depth-First Search (DFS), Depth-Limited Search (DLS), Iterative Deepening Search (IDS), Greedy Search, A* Search, and Generalised A* Search. Problem-specific code was written in `SpecificToProblem.c` and `HashTable.c/h`; the remaining files were kept unchanged as required by the project specification.

File	Role
<code>data_types.h</code>	Global type definitions: State, Node, Queue, enums for ACTIONS and METHODS, Transition_Model.
<code>GRAPH_SEARCH.h</code>	Macro constants and declarations of all public functions used across the project.
<code>GRAPH_SEARCH.c</code>	Entry point (main). Reads user inputs, initialises the root node, dispatches to the chosen algorithm.
<code>SpecificToProblem.c</code>	All problem-specific logic: maze initialisation, state creation, transition model, heuristic, goal test, visualisation.
<code>Standart_Search.c</code>	Generic search infrastructure: frontier management, node expansion, tree traversal loops.
<code>HashTable.c / .h</code>	Open-addressing hash table used as the explored set to avoid revisiting states.

5.2 data_types.h — Global Type Definitions

This header is the single source of truth for all data structures shared across every translation unit. It must be included first by every other file.

5.2.1 enum ACTIONS

Defines the four movement directions available to the agent inside the maze. Each enumerator maps directly to an integer (0–3), which is used as a loop index in the action expansion loop inside the search functions.

```
enum ACTIONS {
    Move_Up,      // 0 - decrease row index
    Move_Down,    // 1 - increase row index
    Move_Left,    // 2 - decrease column index
    Move_Right    // 3 - increase column index
};
```

- `ACTIONS_NUMBER` is set to 4 in `GRAPH_SEARCH.h`, matching this enum exactly.

5.2.2 typedef struct State

Represents a single position in the maze. Two integer fields (row, col) uniquely identify a cell. The float field h_n stores the heuristic value $h(n)$ and is populated only when an informed search algorithm is selected.

```
typedef struct State {
    int    row;    // row index in the MAZE grid
    int    col;    // column index in the MAZE grid
    float  h_n;    // heuristic value h(n); 0 for uninformed searches
} State;
```

5.2.3 enum METHODS

Enumerates all eight search methods. Integer values (1–8) are entered by the user at runtime and matched via a switch statement in main().

5.2.4 Node and Queue Structures

A Node packages a State with its accumulated path cost, the action that produced it, a pointer to its parent, and a child-count used for memory management. A Queue wraps a Node pointer and a next pointer, forming a singly-linked list that serves as the frontier.

```
typedef struct Node {
    State      state;
    float      path_cost;
    enum ACTIONS action;
    struct Node *parent;
    int        Number_of_Child;
} Node;

typedef struct Queue {
    Node      *node;
    struct Queue *next;
} Queue;
```

5.3 GRAPH_SEARCH.h — Constants and Function Declarations

The project-wide header file. It declares every public function and defines the compile-time constants that control maze dimensions and search behaviour.

Macro	Value	Purpose
MAZE_ROWS_20 / MAZE_COLS_20	22	Grid size of the medium maze (20×20 inner cells + 2 border rows/cols).
MAZE_ROWS_50 / MAZE_COLS_50	52	Grid size of the large maze (50×50 inner cells + 2 border rows/cols).
PREDETERMINED_GOAL_STATE	1	When 1, the user is prompted to enter a goal state. When 0, any unvisited cell can be the goal.
ACTIONS_NUMBER	4	Number of possible actions; must match the cardinality of enum ACTIONS.
MAX_SEARCHED_NODE	1,000,000	Upper bound on goal tests to prevent infinite loops in unsolvable mazes.

5.4 GRAPH_SEARCH.c — Program Entry Point

The main function is the orchestration layer. It does not implement any search logic itself; instead it collects configuration from the user, builds the root node, delegates to the appropriate search function, and displays the result.

5.4.1 User Input and Method Selection

The program prints a numbered menu and reads the user's choice into the method variable (type enum METHODS). Two extra parameters are collected conditionally:

- `Max_Level` — only for Depth-Limited Search; defines the maximum tree depth the agent may explore.
- `alpha` (default 0.5) — only for Generalised A* Search; blends path cost and heuristic:

```
f(n) = (1 - alpha) * g(n) + alpha * h(n)
```

```
alpha=0  -> Uniform Cost | alpha=1  -> Greedy | alpha=0.5 -> A*
```

5.4.2 Root Node Initialisation

The root node is declared on the stack. Its fields are set to neutral values before the initial state is created, so that maze selection happens inside `Create_State()` (first call) through the `CURRENT_MAZE` global flag.

```
root.parent      = NULL;
root.path_cost   = 0;
root.action      = NO_ACTION;
root.Number_of_Child = 0;
root.state       = *(Create_State()); // prompts for maze + start cell
```

5.4.3 Goal State and Heuristic Initialisation

When `PREDETERMINED_GOAL_STATE` is 1, `Create_State()` is called a second time to obtain the goal position. For informed searches (Greedy, A*, Generalised A*), the root node's heuristic value is pre-computed and the goal's heuristic is set to 0.

5.4.4 Algorithm Dispatch (switch statement)

A switch on method routes execution to one of three search-tree functions:

- `First_GoalTest_Search_TREE` — BFS and Greedy Search (goal test at generation time, FIFO/priority queue).
- `DepthType_Search_TREE` — DFS, DLS, IDS (LIFO frontier; level cap for DLS/IDS).
- `First_InsertFrontier_Search_TREE` — UCS, A*, Generalised A* (goal test at expansion time, priority queue).

For IDS, the dispatch is wrapped in a loop that increments the depth limit from 0 until a goal node is returned.

5.5 SpecificToProblem.c — Problem-Specific Logic

This is the only file that was written entirely from scratch for the maze problem. It contains all domain knowledge: how the maze is represented, how the agent moves, and how closeness to the goal is estimated.

5.5.1 Maze Representation

The maze is stored in a global 2-D integer array `MAZE[MAZE_ROWS_50][MAZE_COLS_50]`. Each cell holds either 1 (wall) or 0 (free passage). Two mazes are provided:

- Maze 1 (20×20): Hard-coded in a static constant array `MAZE_1_DATA[22][22]`. The outer border is all walls; the interior contains a structured corridor pattern that guarantees connectivity.
- Maze 2 (50×50): Procedurally generated by `Build_Maze_2()`. The function fills the grid with free cells, draws a solid outer border, then stamps horizontal wall rows every two rows with gaps at alternating column positions. Vertical pillar segments are added between wall rows to increase complexity.

5.5.2 Init_Maze(int maze_type)

Copies the chosen maze data into the global `MAZE` array and sets `MAZE_ROWS`, `MAZE_COLS`, and `CURRENT_MAZE`. A static flag prevents `Build_Maze_2()` from running more than once.

5.5.3 Create_State()

Prompts the user to select a maze (on the first call only) and then enter a row/column pair. Input is validated in a loop: out-of-bound coordinates and wall cells are rejected with an informative message. Returns a heap-allocated `State`.

```
while (1) {
    scanf row, col;
    if (out of bounds)        -> continue;
    if (MAZE[row][col] == 1)  -> continue; // wall
    break;                   // valid free cell
}

state->row = row; state->col = col; state->h_n = 0;
```

5.5.4 Result(parent_state, action, trans_model)

The transition function. Computes the new (row, col) after applying action, checks grid bounds, and rejects wall cells. If the move is legal, it writes the new state and a step cost of 1.0 into `trans_model` and returns `TRUE`.

```
Move_Up    -> new_row--
Move_Down  -> new_row++
Move_Left  -> new_col--
Move_Right -> new_col++

// if new position out of bounds or MAZE[new_row][new_col]==1 -> return FALSE
// else -> fill trans_model, step_cost = 1.0, return TRUE
```

5.5.5 Compute_Heuristic_Function(state, goal)

Implements the Manhattan distance heuristic:

```
h(n) = |state.row - goal.row| + |state.col - goal.col|
```

Manhattan distance is admissible for a grid where only orthogonal moves are allowed and each step costs 1, because it never overestimates the true cost. It is also consistent (monotone), guaranteeing that A* will not re-expand any node.

5.5.6 Goal_Test(state, goal_state)

Returns TRUE when the agent's current cell matches the goal cell. It delegates to Compare_States(), which encodes both states as strings and compares them — ensuring the test works generically regardless of the State layout.

5.5.7 Visualisation Functions

Print_Maze_Visual

Renders the maze to the console before the search begins, marking the start cell as "S" and the goal cell as "G", walls as "#", and free cells as ".".

Print_Maze_With_Solution

Called after a successful search. It traces the solution path back from the goal node to the root via parent pointers, marking each visited cell in a local 2-D boolean array. The maze is then re-printed with "*" on every path cell.

5.6 Standart_Search.c — Generic Search Infrastructure

Contains all algorithm-agnostic machinery: frontier management, node expansion, and the three search-tree loops. None of this code references maze-specific details.

5.6.1 First_GoalTest_Search_TREE (BFS, Greedy)

Used by BFS and Greedy Search. The goal test is applied at the moment a child node is generated (early test), which allows BFS to return the shallowest solution without ever inserting the goal into the frontier. Frontier insertion policy:

- BFS: Insert_FIFO — guarantees level-order expansion.
- Greedy: Insert_Priority_Queue_GreedySearch — orders by $h(n)$ ascending.

5.6.2 First_InsertFrontier_Search_TREE (UCS, A*, Generalised A*)

Used by cost-sensitive algorithms where the goal test must be deferred until a node is popped (late test), ensuring optimality. When a child's state is already in the frontier with a higher cost, the old node is removed and the cheaper child is inserted. Insertion policies:

- UCS: orders by $g(n)$ (path cost) ascending.
- A*: orders by $f(n) = g(n) + h(n)$ ascending.
- Generalised A*: orders by $f(n) = (1-\alpha) \cdot g(n) + \alpha \cdot h(n)$ ascending.

5.6.3 DepthType_Search_TREE (DFS, DLS, IDS)

Uses a LIFO frontier (stack). For DLS and IDS, nodes at the depth limit are not expanded; they are cleared from memory via `Clear_All_Branch()`. The three node counters are declared static, so for IDS they accumulate across all iterative calls — reported totals reflect the entire search, not just the final iteration.

5.6.4 Child_Node(parent, action)

Calls `Result()` and, if the move is legal, allocates a new Node on the heap, fills all fields, increments the parent's child count, and returns the pointer. Returns NULL for illegal moves.

5.6.5 Frontier Management

The frontier is a singly-linked list. Key operations:

- `Start_Frontier`: allocates the first Queue cell containing the root.
- `Pop`: removes and returns the head node, freeing the Queue wrapper.
- `Insert_FIFO / Insert_LIFO`: append to tail or prepend to head.
- `Insert_Priority_Queue_*`: linear-scan insertion maintaining sorted order.
- `Frontier_search`: linear scan returning the node whose state matches; used for duplicate detection.
- `Remove_Node_From_Frontier`: unlinks a Queue cell by updating the predecessor's next pointer.

5.6.6 Memory Management

`Clear_Single_Branch` frees one node and decrements its parent's child count. `Clear_All_Branch` does this recursively upward until it reaches a node that still has living children or the root — preventing memory accumulation during DLS/IDS.

5.6.7 Show_Solution_Path and Print_Node

`Show_Solution_Path` traverses the parent chain from goal to root, printing each state and the action that produced it, then reports the total path cost. `Print_Node` is a debug helper that formats a single node for console output.

5.7 HashTable.c / HashTable.h — Explored Set

Implements an open-addressing hash table with linear probing and dynamic resizing. It is used as the explored set in all search functions to detect and skip already-visited states.

5.7.1 Configuration Constants (HashTable.h)

- `HASH_TABLE_BASED_SIZE` = 29 — initial table capacity (prime number).
- `HASH_TABLE_INCREASING_RATE` = 70 — resizing threshold: when load factor exceeds 70%, the table doubles in size.
- `MAX_KEY_SIZE` = 16 — maximum length of a key string.

5.7.2 Generate_HashTable_Key(state, key)

The only problem-specific function in this file. Encodes a State as the string "row_col" using sprintf. Because (row, col) uniquely identifies a maze cell, this key guarantees that no two distinct states produce the same string.

```
void Generate_HashTable_Key(const State *const state, unsigned char* key)
{
    sprintf((char*)key, "%d_%d", state->row, state->col);
    // e.g. state at row=3, col=7 -> key = "3_7"
}
```

5.7.3 Hashing and Collision Resolution

The hash function uses a polynomial rolling scheme with base $a = 151$ (a prime larger than the ASCII alphabet size). Collisions are resolved by linear probing: if the computed slot is occupied, the index is incremented circularly until an empty slot is found.

```
// hash = SUM( a^(len-i-1) * key[i] ) mod table_size
```

5.7.4 Insert and Search

ht_insert generates the key for a State and calls ht_insert_key, which first checks the load factor and resizes if necessary, then finds the next free slot via linear probing and stores the key string. ht_search hashes the query key, then probes until either the key is found (returns TRUE) or an empty slot is encountered (returns FALSE).

5.7.5 Resize_Hash_Table

Creates a new, larger table (next prime after $2 \times$ current size), re-inserts all existing keys, then swaps the internal arrays between the old and new table objects before freeing the temporary structure. This keeps the external pointer valid.

5.8 Interaction Between Files — Call Flow Summary

The table below summarises how the six files collaborate during a single search run.

Step	Caller	Callee	Description
1	main() / GRAPH_SEARCH.c	Create_State() / SpecificToProblem.c	Selects maze, prompts for start/goal positions.
2	main()	Search function / Standart_Search.c	Dispatches to BFS / DFS / A* etc. with root node.
3	Search function	Child_Node() / Standart_Search.c	Expands current node for each action.
4	Child_Node()	Result() / SpecificToProblem.c	Computes new state, validates bounds and walls.
5	Search function	ht_insert / ht_search / HashTable.c	Adds expanded states to explored set; checks for revisits.
6	Search function	Goal_Test() / SpecificToProblem.c	Checks whether the current state equals the goal.
7	main()	Show_Solution_Path() / Standart_Search.c	Traces parent chain and prints the solution.
8	main()	Print_Maze_With_Solution() / SpecificToProblem.c	Renders maze with solution path marked as "***".

5.9 Key Design Decisions

Separation of generic and problem-specific code

The framework strictly separates problem-specific logic (SpecificToProblem.c) from generic search machinery (Standart_Search.c). Adding a new search domain requires rewriting only SpecificToProblem.c and updating the constants in data_types.h and GRAPH_SEARCH.h — the search loops, frontier, and hash table are fully reusable.

Admissible and consistent heuristic

Manhattan distance satisfies both admissibility (never overestimates) and consistency ($h(n) \leq c(n, n') + h(n')$ for every successor n') on a unit-cost grid. This guarantees that A* finds the optimal path without re-expanding nodes.

Generalised A* with parameter alpha

The generalised evaluation function $f(n) = (1-\alpha) \cdot g(n) + \alpha \cdot h(n)$ interpolates between UCS ($\alpha=0$) and Greedy Search ($\alpha=1$). Intermediate values trade off optimality against speed: larger alpha expands fewer nodes but may miss the shortest path.

Dynamic hash table resizing

The explored set resizes automatically when the load factor exceeds 70%, preventing performance degradation due to clustering. Capacity always grows to the next prime number, maintaining good hash distribution.

Memory management in depth-based searches

DLS and IDS can produce very large trees. Nodes pruned at the depth limit are freed immediately via Clear_All_Branch(), and the recursive upward sweep removes parent nodes that have lost all children — keeping memory usage bounded.

6. Conclusion and Evaluation

6.1 Overall Assessment

In this project, the shortest-path-in-maze problem was successfully formalized as a state-space search problem and solved with eight different algorithms covering both uninformed (BFS, UCS, DFS, DLS, IDS) and informed (Greedy, A*, Generalised A*) strategies. The agent's state was reduced to a (row, column) pair, four unit-cost movement actions were defined, and Manhattan distance was adopted as an admissible and consistent heuristic. Two mazes of significantly different scales (20×20 manually designed and 50×50 algorithmically generated) were used so that algorithmic behavior could be observed across both small and large search spaces.

The simulation results confirm the theoretical expectations of each algorithm. BFS, UCS, A* (Manhattan) and Generalised A* with $\alpha \in \{0.0, 0.5\}$ all returned the optimal path on every test scenario (e.g. 58 steps for S1 and 142 steps for S4), while DFS, Greedy, and Generalised A* with $\alpha = 1.0$ produced sub-optimal solutions in several scenarios (DFS reached up to 235 steps in S5 versus the optimal 189). The cost of optimality is visible in the node-generation counts: BFS and UCS expanded the largest number of nodes (up to 2852 on S5), whereas A* with the Manhattan heuristic reduced this count by roughly 35% on the 50×50 maze, and Greedy/Generalised A* ($\alpha = 1.0$) reduced it even further at the price of optimality.

Taken together, these observations support the conclusion drawn during the analysis: A* with Manhattan distance is the most balanced algorithm for this problem, offering the same optimal path as BFS while expanding only about 60–65% of the nodes that BFS expands. The Generalised A* parameter α confirmed its expected interpolating behavior — at $\alpha = 0$ the algorithm matches UCS exactly, at $\alpha = 1$ it matches Greedy, and at $\alpha = 0.5$ it behaves identically to classical A*. The project therefore meets all of its stated learning objectives: formal problem modeling, heuristic design with admissibility/consistency proofs, software implementation in C, and comparative experimental evaluation.

6.2 Challenges and Solutions

Several practical challenges were encountered during the implementation and testing phases. The most important ones, together with the adopted solutions, are summarized below.

- Hash table capacity for the large maze: The default explored-set capacity inherited from the Romania-map example was too small for the 50×50 maze. `HASH_TABLE_BASED_SIZE` was raised to a larger prime (2503) and the dynamic-resize policy (70% load factor → double in size) was preserved so that the table grows automatically when needed without manual tuning per maze.
- Hash key length: The original key generation routine produced fixed-width city-number strings. For maze coordinates the longest possible key on the 50×50 grid is "49_49" (5 characters plus terminator). `MAX_KEY_SIZE` was set to 12 to leave a safe margin and to keep `snprintf`-based key generation robust against off-by-one errors.
- Generalised A* integration: `First_InsertFrontier_Search_TREE()` originally handled only UCS and A*. A new case was added for `GeneralizedAStarSearch`, and a parameter α was threaded through `main()` into the search function and from there into `Insert_Priority_Queue_GENERALIZED_A_Star()`. Care was taken to keep the existing function signatures of UCS and A* unchanged so that no regression was introduced in the previously working algorithms.
- Optimality verification for cost-sensitive algorithms: Because every step has cost 1.0, BFS and UCS produce identical path lengths. This was used as a sanity check: any deviation between BFS and UCS would indicate a bug. The same check was extended to A* and Generalised A* ($\alpha \leq 0.5$), all of which match BFS on every scenario, confirming that the priority queue and the duplicate-detection logic operate correctly.
- Maze visualization and solution rendering: Two helper functions, `Print_Maze_Visual()` and `Print_Maze_With_Solution()`, were written to make experiments inspectable. The latter walks the parent chain from the goal back to the root, marks visited cells in a local 2-D flag array, and re-prints the maze with the path overlaid — which proved essential for debugging Greedy and DFS, where the produced paths are not always intuitive.

6.3 Future Work

Although the current implementation already covers a representative set of search algorithms, several directions could extend the project further:

- Larger and more varied mazes: Performance can be measured on 100×100 or non-square mazes generated by classical procedures such as recursive backtracking or Prim's algorithm, in order to study how each algorithm scales as branching density and corridor length grow.
- Alternative heuristics: Chebyshev distance and diagonal distance can be implemented and compared, especially in variants of the problem that allow 8-directional movement. Weighted-A* variants ($f = g + w \cdot h$ with $w > 1$) could also be benchmarked to quantify the speed-versus-optimality trade-off more finely than the current α sweep.
- Bidirectional A*: Searching simultaneously from both the start and the goal and meeting in the middle is expected to reduce the number of expanded nodes substantially on long-corridor mazes such as Maze 2, and would be a natural next step.
- Non-uniform terrain costs: Extending the maze from binary walls to weighted terrain (plains, hills, mud, etc.) — which corresponds to Topic 5 in the project list — would make UCS and A* differ in path cost, providing a richer experimental setting.
- Empirical complexity analysis: Running each algorithm over many randomly generated start-goal pairs and reporting averaged generated-node counts, memory footprint, and wall-clock time would allow tighter empirical confidence intervals to be reported instead of single-run measurements.

6.4 References

- [1] S. J. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed., Pearson, 2020.**
- [2] Course notes — Introduction to Artificial Intelligence, Yıldız Technical University, Department of Computer Engineering, Spring 2026.**
- [3] Project specification — IAI 2026 Project: Topic 4, "The Shortest Path in Maze".**
- [4] Wikipedia, "A* search algorithm", https://en.wikipedia.org/wiki/A*_search_algorithm**
- [5] Wikipedia, "Maze generation algorithm", https://en.wikipedia.org/wiki/Maze_generation_algorithm**
- [6] Wikipedia, "Manhattan distance", https://en.wikipedia.org/wiki/Taxicab_geometry**
- [7] R. Sedgewick and K. Wayne, Algorithms, 4th ed., Addison-Wesley, 2011 — chapters on graph search and priority queues.**
- [8] Math Is Fun, "Mazes", <https://www.mathsisfun.com/measure/mazes.html> (reference page cited in the project specification).**